

Assessment Cover Sheet

Programme Name:	FdSc / Cert HE Computing
Semester 1 or 2:	2
Module Code:	S2-PFD200
Due Date:	12.06.2026
Assessment Type:	Portfolio
Name	Vladislavs Vigulis

Part 1:Self-Assessment

A self-assessment helps me see what skills I already have learned and what I still need to change. As computing student, I have learned various important Ideas during my studies, especially in programming. At the same time, I know there are still lot of thing where I need to develop more knowledge and practical experience if I want to reach my future goals.

One of my strongest areas at the minute is my basic programming cognition. I have activity with programming languages like Java and Python during my class. Through various assignments and exercises, I have learned how to use changeable, loops, conditions, and purpose to solve problems. I also understand the base idea of object-oriented programming, consider how classes and methods work. These idea have helped me realize how programs are structured and how various parts of a program work together.

Some other skill I have improved is problem solving. Programming infrequently requires break a problem down into smaller steps. I have been practicing this while completing coding project, also seemly more confident in reading and understanding code. fixing errors when programs do not run right. Debugging can sometimes be discouraging, but it has helped me learn how to weighing more logically when writing code.

During my studies, I have also been present to different software development approaches, such as Waterfall and Agile. Studying about these methods assist me understand how software is planned and developed in realistic organization. Today I understand that software development is not just about writing code, but also about preparation, testing, and working with a [group](#).

I have gained some useful skills, there are still various areas where I need to improve. One area is data structures and

algorithms. I understand primary structures like arrays and lists, but I know that I need more practice with more complex concepts and writing efficient code. This is thing I would like to improve because it is an important skill for [programmers](#).

Some other area I would like to grow is web development. I have some basic Understanding of HTML and JavaScript. I have not yet made-up a full web application on my own. Learning how front-end and back-end systems work in concert would be very useful for building real [projects](#).

My long-term object is to work as a software developer. I would like to build applications wich are useful and can solve real problems. To scope this goal, I need to continue practicing programming and educat new applications. also want to work on more personal projects that I can increase practical [education](#).

General, I believeThat I have a good starting point in computing,.especially with programming basics. Still, I understand that there is still a lot to learn. By continuing to improve my technical skills and gaining more hands-on experience, I can prepare myself better for a future career in the technology field.

Part 2 Project Proposal and Negotiation

Project Title

SecurePass Toolkit: Password Generator and Strength Checker

Project Overview.

The purpose of this mini project is to develop a Python application that helps users create stronger passwords and evaluate password strength. The project supports both programming skill development and cybersecurity sentience. The SecurePass Toolkit is a software application that helps users create and manage secure passwords. Many people use weak passwords or apply the same password crosswis with accounts, which increases the risk of cyberattacks. This

project aims to help users improve their password security through several useful tools.

Application will be developed using Python and will view conception such as password generation, password strength checking, password hashing, and secure keeping of passwords.

The task will indefinite quantity practical experience in cybersecurity and secure software improvement while creating a useful application for everyday users.

Project Objectives

Important objective in this project is to develop a secure password management application that helps users make, assess, and store strong passwords safely.

Specific objectives include:

- Designing and develop the SecurePass Toolkit using Python.**

- Create a secure password generator** that produces strong and random passwords.

- Apply a password strength checker** that evaluates password security supported on length, complexity, and character variety.

- apply password hashing techniques** to defend user passwords from unauthorized access.

- implement encryption methods** for secure storage of sensitive data.

- develop a user-friendly interface** that allows users to easily access the application's features.

- test the application** to ensure practicality, reliability, and security.

- improve knowledge and practical skills in cybersecurity**, particularly in password security and secure software program development.

- gain experience in the software development lifecycle**, including planning, design, implementation, testing, and documentation.

- create a portfolio project** that demonstrates programming, cybersecurity, and problem-solving skills.

Project Description

SecurePass Toolkit allows the user to select between two main options. The first option generates a secure password using a chosen section and selected attribute types such as capital letters, lowercase letters, numbers, and symbolization. The 2nd option checks the strength of a password and returns a grade, rating, estimated entropy, and feedback for improvement.

Performing was created using Python's standard library. The ``secrets`` module was used instead of ``random`` because it provides stronger security for password generation. Day-to-day expressions were used to study password form and character variety.

Features Implemented.

The **SecurePass Toolkit** includes the following features:

Password Generator.

- Generates strong and random passwords.

- Users can choose the password size .

- Includes letters, numbers, and special characters.

Password Strength Checker.

- Checks whether a password is weak, medium, or strong.

- Gives feedback to help users create better passwords.

Password Hashin.

- Converts passwords into a secure format before storing them.

- Helps protect passwords from being viewed by others.

Secure Password Storage.

- Stores passwords safely in a database.

- Uses encryption to improve security.

Login System

- Allows users to create an account and log in.

- Ensures only authorized users can access stored passwords.

User-Friendly Interface

- Simple and easy-to-use design.

- Makes it easy for users to access all features.

Input Validation

Checks user input for errors.

Prevents invalid data from being entered into the system.

Summary

SecurePass Toolkit supply users with tools to generate strong passwords, check password strength, and store passwords securely. These attributes help improve password security while demonstrating important cybersecurity and programming skills.

Testing Summary

Testing was carried out to ensure that all features of the **SecurePass Toolkit** worked correctly and met the project requirements.

Password Generator Testing

Tested different password lengths and character options.

Verified that generated passwords were random and met the selected criteria.

Result - **Passed**.

Password Strength Checker Testing

Tested weak, medium, and strong passwords.

Verified that the system correctly rated password strength.

Result - **Passed**.

Login System Testing

Tested user registration and login with valid and invalid credentials.

Verified that only registered users could log in.

Result - **Passed**.

Password Storage Testing

Tested saving and retrieving passwords from the database.

Verified that passwords were stored securely.

Result - **Passed**.

Input Validation Testing

Tested empty fields and incorrect inputs.

Verified that appropriate error messages were displayed.

Result - **Passed**.

Overall Results

All leading features were tested successfully and functioned as expected. The application was able to make secure passwords, check password strength, store data securely, and manage user logins in effect. Major problems found during investigation were fixed, resulting in a firm and functional system.

Skills Developed

During the development of the **SecurePass Toolkit**, I improved several technical and professional skills:

Programming Skills

- Improved my Python programming skills.

- Learned how to build a complete application with multiple features.

- Gained experience in debugging and fixing errors.

Cybersecurity Skills

- Learned about password security and best practices.

- Gained practical experience with password hashing and encryption.

- Improved my understanding of secure data storage.

Software Development Skills

- Learned how to plan, design, develop, and test a software project.

- Improved my understanding of the Software Development Life Cycle (SDLC).

- Gained experience in documenting a project.

Database Skills

- Knowing how to store and manage data using a database.

- Made-up my knowledge of data organization and retrieval.

Problem-Solving Skills

- Defined logical thinking and problem-solving abilities.

- Knowing how to identify issues and find effective solutions.

Time Management and Project Management

- Improved my ability to manage tasks and meet deadlines.

- Learned how to organize project activities and follow a development plan.

Summary

Generally, project helped me strengthen my programming, cybersecurity, database, problem-solving skills. It also gave me valued experience in developing a secure software application from planning through to testing and pass completion.

Future Improvements

Although the **SecurePass Toolkit** is functional, there are several improvements that could be made in the future:

Password Breach Checker

Add a feature that checks whether a password has been exposed in known data breaches.

Help users avoid using compromised passwords.

Password Manager

Allow users to save and organize multiple passwords for different accounts.

Make password management more convenient.

Two-Factor Authentication

Add an extra layer of security during login.

Improve protection against unauthorized access.

Password Expiry Reminders

Notify users when it is time to update their passwords.

Encourage better password security habits.

5. Improved User Interface

Enhance the design to make the application more attractive and easier to use.

Add themes and customization options.

Cloud Backup

Allow users to securely back up their encrypted passwords online.

Prevent data loss if the device is damaged or lost.

Mobile Version

Develop a mobile application for Android and iOS devices.

Enable users to access their passwords anywhere.

Summary

Future day better will focus on increasing security, improving usefulness, and adding more precocious features. These improve would make the SecurePass Toolkit more practical, secure, and useful for a beamy range of users.

Conclusion

project successfully meets the aim of building a small but useful Python application. It show practical programming ability and links directly to

professional development by rising technical skills and promoting cybersecurity knowing.

The **SecurePass Toolkit** was successfully developed to assist users create and manage secure passwords. Application includes attributes such as a password setup, password strength check, secure password storage, login system.

Project, developed my Python programming, cybersecurity, and problem-solving skills.

also learned more about password security, encryption, and software development.

Overall, the project was a valued learning experience and helped me advantage practical skills that can be useful in future computing and cybersecurity projects. In future, more features can be added to make application even better and more secure.

SecurePass Toolkit

Overview

SecurePass Toolkit is a small Python command-line application that create secure passwords and evaluate password weakness. The project was designed to develop practical Python programming, ability also building awareness of cybersecurity best exercise.

Features

- Generate a secure password using custom settings
- Check the strength of an existing password
- Display a score, rating, entropy estimate, and improvement feedback
- Simple command-line menu system
- Unit tests included

Technologies Used

- Python 3
- Standard library only (``secrets``, ``string``, ``math``, ``re``, ``unittest``)

How to Run

Make sure Python 3 is installed.
Open a terminal in the project folder.
Run:

```
```bash
python main.py
```
```

How to Run Tests

```
```bash
python -m unittest test_main.py
```
```

Project Structure

- `main.py` – main application
- `test\_main.py` – unit tests
- `README.md` – user documentation
- `report.md` – short report for submission

Example Use Cases

- Creating stronger passwords for personal accounts
- Demonstrating Python programming skills in a portfolio
- Showing basic cybersecurity awareness in a mini project

Mini Project Report – SecurePass Toolkit

Project Title

SecurePass Toolkit: Password Generator and Strength Checker

Project Aim

The purpose of this mini project is to develop a Python exercise that helps users create stronger passwords and evaluate password strength. The project supports both programming skill process and cybersecurity cognisance.

Objectives

- Build a working Python command-line application
- Apply midpoint programming concepts such as functions, classes, validation, and error handling
- Demonstrate consciousness of secure password practices
- Produce clear documentation and testing evidence

Mini Project Report – SecurePass Toolkit

Project Description

SecurePass Toolkit allows user to select between two main options. first alternate generates a secure password using a chosen length and selected lineament types such as capital letters, lower-case letter letters, numbers, and symbols. The second option checks the strength of a password and returns a score, rating, calculation information, and feedback for condition.

The method is created using Python's standard library. The ``secrets`` ability was used instead of ``random`` because it provides stronger security for password generation. Regular expressions were used to canvass password design and lineament motley.

Features Implemented

- Menu-based interface
- Secure password generation
- Password strength analysis
- Feedback messages to improve weak passwords
- Input validation and exception handling
- Unit testing

Testing Summary

The request was tested using Python's ``unittest`` module. Test cases checked the following:

- Passwords are generated at the requested length
- Generated passwords include the required character types
- Weak passwords are identified correctly
- Strong passwords are classified appropriately

Skills Developed

This project helped develop the following skills:

- Python programming
- Use of built-in libraries
- Writing test cases
- Problem solving and debugging
- Documentation and software presentation
- Base understanding of cybersecurity good practice

Future Improvements

- Add a graphical user interface using Tkinter
- Save generated password reports to a file
- Add dictionary-based checks for compromised password patterns
- Allow export of analysis results

Conclusion

This task successfully achieved its aim of developing a small but useful Python application called **SecurePass Toolkit**. The application was designed to help users make strong passwords and evaluate the strength of existing passwords, upgrade better password security and cybersecurity knowing. All of the main objectives were realized, exclude secure password generation, password strength investigation, input signal validation, and testing.

End-to-end the project, I applied a scope of programming conception learned during my studies, such as functions, conditional statements, loops, error handling, and the use of Python libraries. This allowed me to strengthen my practical programming skills and gain confidence in developing a complete software application from planning and design through to implementation and testing.

Project also improved my knowing of cybersecurity principles. I learned why strong passwords are important, how password strength can be measured, and how secure password generation helps protect users from common cyber threats. By using Python's secrets module, I increase experience in applying security-focused programming activity rather than relying on less secure know-how.

In addition, the project helped me develop essential problem-solving and debugging skills. During development, I convergence challenges that needed testing, troubleshooting, and improving the code to secure the application works correctly. This experience increased my ability to identify issues and find effective statement.

General, the SecurePass Toolkit show practical programming ability and shows how computing skills can be practical to solve real-world security trouble. The project has bring to my professional development by rising my technical knowledge, programming hopefulness, and understanding of cybersecurity best recitation. In the future, additional features such as a graphic user program, password storage, encryption, and two-factor authentication could be added to further enhance the functionality and security of the application.

References

Bishop, M. (2018) *Computer Security: Art and Science*. 2nd edn. Boston: Addison-Wesley.

Menezes, A.J., van Oorschot, P.C. and Vanstone, S.A. (2018) *Handbook of Applied Cryptography*. Boca Raton: CRC Press.

National Cyber Security Centre (NCSC) (2023) *Password Guidance: Simplifying Your Approach*. Available at: <https://www.ncsc.gov.uk> (Accessed: 4 June 2026).

Python Software Foundation (2026) *Python Documentation*. Available at: <https://docs.python.org/3/> (Accessed: 4 June 2026).

Python Software Foundation (2026) *secrets — Generate Secure Random Numbers for Managing Secrets*. Available at: <https://docs.python.org/3/library/secrets.html> (Accessed: 4 June 2026).

Stallings, W. (2017) *Cryptography and Network Security: Principles and Practice*. 7th edn. Harlow: Pearson.

OWASP Foundation (2024) *Authentication Cheat Sheet*. Available at: <https://owasp.org> (Accessed: 4 June 2026).

Sweigart, A. (2019) *Automate the Boring Stuff with Python*. 2nd edn. San Francisco: No Starch Press.

In-text citation examples

Passwords should be long and complex to improve security (NCSC, 2023).

The Python `secrets` module is recommended for generating cryptographically secure random values (Python Software Foundation, 2026).

Password hashing helps protect stored credentials from unauthorized access (Stallings, 2017).

Secure authentication practices reduce cybersecurity risks (OWASP Foundation, 2024).

<https://github.com/vigulisvladislavs-cmd/task-manager-project>

<https://github.com/vigulisvladislavs-cmd/task-manager-project>